

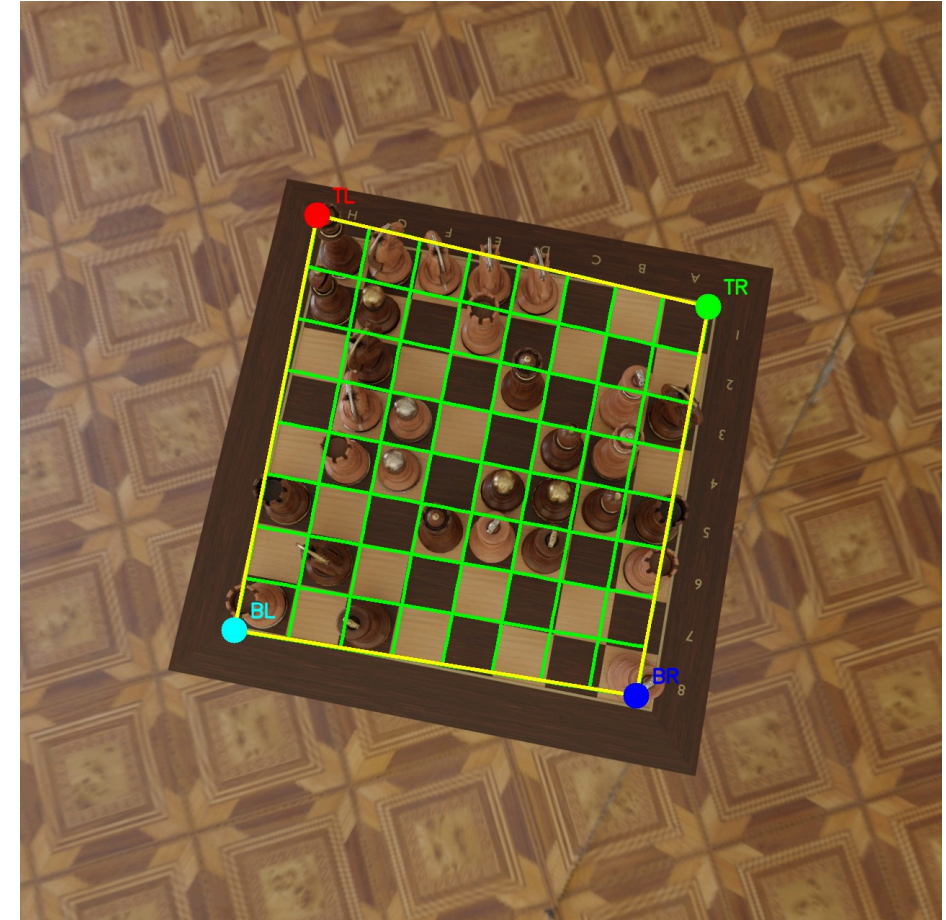
Análise de Tabuleiros de Xadrez

Andamento do projeto — Classificação de Peças com Deep Learning

Davi Ludvig e Julia Macedo

Disciplina: INE410121 / TRV410001 — Visão Computacional - UFSC

Dataset: Synthetic Chess Board Images — Kaggle (thefamousrat)



O que foi feito neste período

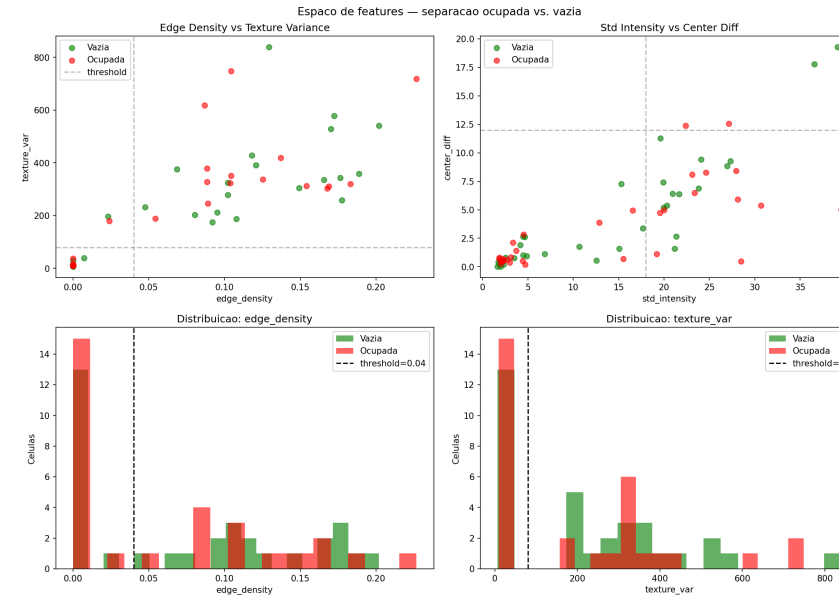
A pipeline clássica (apresentação anterior) detectava **ocupação** e **cor** — mas não o tipo de peça. Este andamento cobre a implementação completa do classificador.

Etapa	Status anterior	Status atual
Leitura do tabuleiro (Hough + homografia)	✅ Concluído	✅ Concluído
Deteção de ocupação (votação de features)	✅ Concluído	✅ Concluído
Classificação de cor (HSV)	✅ Concluído	✅ Concluído
Identificação de tipo de peça	⏳ Pendente	✅ Concluído — F1 = 91%
Notação PGN / deteção de jogadas	📋 Planejado	📋 Planejado

O Desafio da Classificação de Tipo

As peças no dataset são feitas de **madeira com tons similares** ao tabuleiro, o que dificulta abordagens puramente clássicas:

- Template matching: sensível à escala e rotação
- HOG / Hu Moments: confusão entre peças de silhueta parecida (peão × bispo)
- A câmera em ângulo deforma a silhueta das peças



Feature space mostra grande sobreposição — desafio inerente ao dataset.

Solução: Transfer Learning com ResNet-34

Optamos por **Deep Learning** para a classificação de tipo, mantendo o pipeline clássico para tudo que o antecede.

Por que ResNet-34?

- Conexões residuais → sem gradiente evanescente
- Pré-treinada no ImageNet → features de bordas e texturas já aprendidas
- Tamanho moderado → treina bem com ~48 000 células

Estratégia de duas fases:

1. **Transfer Learning** — backbone congelado, só a cabeça FC treina
2. **Fine-tuning** — backbone descongelado com LR discriminativo por camada

```
ImageNet → ResNet-34
↓ backbone (congelado → descongelado)
[conv1][layer1][layer2][layer3][layer4]
↓ avgpool + flatten
[FC: 512 → 12 classes]
↓
pawn_w pawn_b rook_w rook_b
knight_w knight_b bishop_w bishop_b
queen_w queen_b king_w king_b
```

Construção do Dataset de Treinamento

Para treinar, precisamos de **imagens rotuladas de células individuais**:

1. Para cada imagem do dataset, aplica-se a homografia GT para endireitar o tabuleiro
2. Detecta-se a orientação automaticamente (densidade de bordas)
3. Cada célula ocupada é salva em

`outputs/piece_cells/{label}/`

Volume: $\sim 1\,943$ imagens $\times$ ~ 25 peças $\approx$ **48 000 células**

Classe	Qtd aprox.
pawn_w / pawn_b	$\sim 9\,000$ cada
rook_w / rook_b	$\sim 2\,100$ cada
knight_w/b	$\sim 2\,100$ cada
bishop_w/b	$\sim 2\,100$ cada
queen_w/b	$\sim 1\,100$ cada
king_w/b	$\sim 1\,100$ cada

Peões são $\sim 3\times$ mais frequentes que reis — classes **desbalanceadas**.

Treinamento — Detalhes

Fase 1 — Transfer Learning (10 épocas)

Backbone	congelado
Otimizador	Adam · LR 1e-3
Loss	CrossEntropy
Scheduler	CosineAnnealing

Fase 2 — Fine-tuning (15 épocas)

Backbone	descongelado
Otimizador	AdamW · wd 1e-4
LR	discriminativo por camada (fator 0.3×)
Loss	CrossEntropy + label smoothing 0.1
Scheduler	warmup 2ep + cosine annealing
Grad clip	1.0

Augmentations:

```
RandomHorizontalFlip(p=0.5)
RandomVerticalFlip(p=0.5)
RandomRotation(15°)
ColorJitter(brightness=0.3, contrast=0.3,
            saturation=0.2)
Normalize(ImageNet  $\mu/\sigma$ )
```

Device: CUDA · Batch: 64 · img_size: 224×224

Resultados — Pipeline Clássica (ocupação)

Avaliado em 10 imagens com cantos detectados automaticamente (Hough):

Métrica	Valor
Acurácia média	65.5%
Precisão média	62.5%
Recall médio	81.6%
F1 médio	69.8%

Observação: recall alto (81.6%) — a maioria das peças é detectada, mas há falsos positivos. O principal gargalo é a detecção automática de cantos: com cantos GT, F1 sobe para **88%**. Material uniforme (madeira/madeira) é o fator limitante intrínseco.

Melhor imagem: F1 = 98.4% | Pior imagem: F1 = 50.0%

Resultados — Classificador DL (tipo de peça)

Avaliado em 50 imagens com ocupação GT (isola o classificador do pipeline clássico):

Métrica	Valor
Precisão	91.0%
Recall	91.0%
F1	91.0%
TP corretos / errados	1 412 / 140

Classe	Acurácia
knight_b / knight_w	96.4% / 95.9%
pawn_b / pawn_w	96.3% / 93.8%
rook_b / rook_w	89.2% / 90.1%
queen_b / queen_w	87.8% / 90.8%
bishop_b / bishop_w	88.2% / 87.6%
king_w / king_b	89.7% / 84.5% ← mais difícil

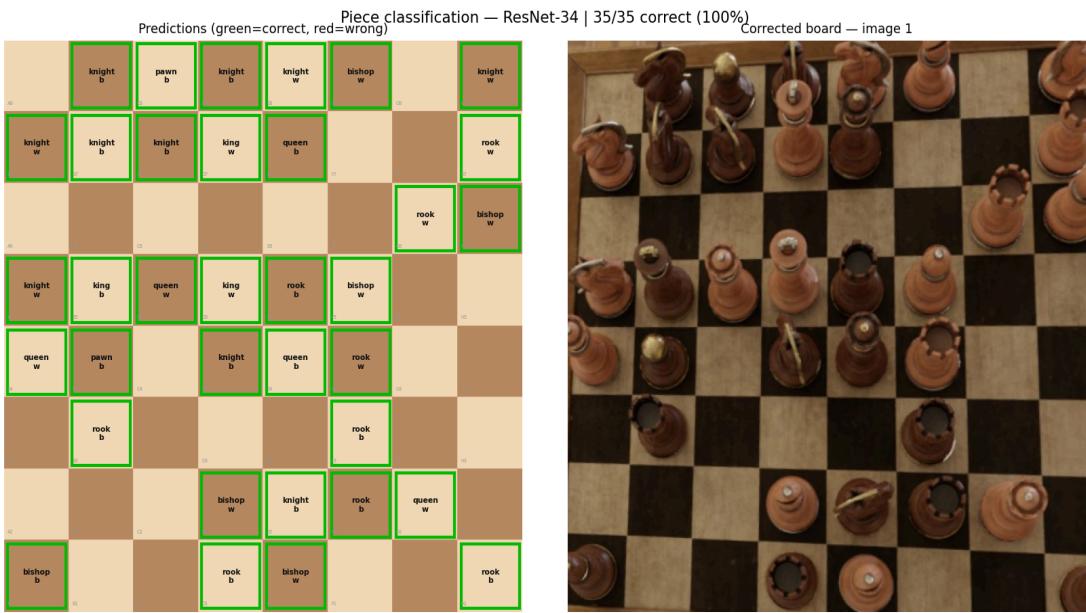


Imagem 1 — 35/35 corretas (100%)

Pipeline Completa Atual

```
Imagem original (1280×1280)
  ▼ Hough + homografia (clássico)
Tabuleiro retificado (480×480)
  ▼ Votação de features (clássico)
Mapa de ocupação 8×8
  ▼ ResNet-34 transfer learning (DL)
Mapa de peças: {A1: pawn_w, E4: queen_b, ...}
  ▼ Comparação temporal (clássico)
Detecção de jogadas
```

Acurácia de tipo+cor (pipeline completa): $F1 \approx 91\%$ para o classificador DL; $F1 \approx 70\%$ para ocupação (pipeline completa) / 88% com cantos GT.

FEN Notation — Como Funcionaria

O `piece_map` já tem tudo que o FEN precisa:

```
piece_map = {"A8":"rook_b", "E1":"king_w", ...}

FEN_SYM = {
    "pawn_w":"P", "rook_w":"R", "knight_w":"N",
    "bishop_w":"B", "queen_w":"Q", "king_w":"K",
    "pawn_b":"p", "rook_b":"r", "knight_b":"n",
    "bishop_b":"b", "queen_b":"q", "king_b":"k",
}
# rank 8→1, file A→H, vazios = número
# "r1bqkb1r/pppp1ppp/2n5/4p3/..."
```

Detecção de jogada (dois frames):

```
emptied = {sq for sq in map_t if sq not in map_t1}
filled  = {sq for sq in map_t1 if sq not in map_t}
# from_sq → to_sq  ex: E2 → E4  =  "e2e4"
```

Limitação: dataset tem >32 peças por imagem (sintético, posições inválidas). Move notation requer frames de um jogo real contínuo.

O que faltaria implementar:

- Detectar roque (rei move 2 casas)
- En passant (peão captura diagonal sem peça no destino)
- Promoção (peão chega à última fileira)
- Validação de legalidade do lance

Próximos Passos

FEN / Notação de jogadas

- Conversão `piece_map` → FEN é direta (ver slide anterior)
- Para jogadas reais: coletar frames de um jogo contínuo
- Validar legalidade dos lances contra as regras do xadrez

Melhorar a pipeline clássica

- Reduzir falsos positivos no detector de ocupação
- Resolver ambiguidade de orientação 180° sem GT

Distribuição do modelo

- Publicar `.pth` (GitHub Releases ou HuggingFace Hub)
- Atualizar `setup.py` para baixar automaticamente

Domain shift

- Avaliar performance em tabuleiros reais (não sintéticos)
- Iluminação variável, peças de diferentes materiais

Conclusão

Componente	Abordagem	Resultado
Detecção do tabuleiro	Hough + Homografia	Robusto
Segmentação 8×8	Divisão uniforme	Exato
Ocupação	Votação de features	F1 = 70% (ponta a ponta)
Cor da peça	Threshold HSV	~88%
Tipo da peça	ResNet-34 (TL + FT)	F1 = 91%

O projeto demonstra como **combinar visão clássica com transfer learning** para construir um pipeline completo de leitura de tabuleiro, aproveitando o melhor de cada abordagem.